Karari Beauty

Performance optimisation & security audit - before / after report

Repository: mithilmohite27/karari-beauty-website | Branch: main | Commit: bd90550 | 5 August 2026

Executive summary

The client reported the site loading slowly. Measurement against the live site identified **three independent causes**, not one. In order of impact: product images were bypassing image optimisation entirely and were being served as ~2 MB PNG originals; every public page was configured so the CDN was never used, forcing a full server render and four database queries on every single request; and several static assets were dramatically oversized, including a 240 KB favicon downloaded on every page view.

All three are fixed and verified. Alongside this, an audit of the payment and admin layers found four defects - one of which silently corrupted order state - and these are also fixed. **Two launch blockers remain and require action from the team; they are listed in section 7.**

1. Headline results

Measure	Before	After	Change
Homepage response time	858 - 1,190 ms	31 - 77 ms	**~25x faster**
Collection page response	618 - 1,199 ms	37 - 223 ms	**~10x faster**
Vercel CDN status	MISS on every request	HIT / PRERENDER	**CDN now used**
Page cache-control	private, no-store	Edge-cached, ISR	-
Images via optimiser	0 of 73 (opted out)	102 of 102	**100% coverage**
Avg product image weight	1,995 KB	63 KB	**-97%**
Raw unoptimised images	73 on homepage	0	**eliminated**
Static assets total	5,214 KB	1,480 KB	**-72%**
Favicon (served raw)	240 KB	2 KB	**-99%**
Security headers	None	4 applied	-
Page HTML weight	974 KB	1,118 KB	not improved

How these were measured. Both columns were captured against the live production site at www.kararibeauty.com - 'before' prior to any change, 'after' following deployment of commit bd90550. Response times are the range across four consecutive cache-busted requests. Image weight is the mean of an eight-image sample fetched from the live page.

Two honest caveats. First, the 63 KB image average is measured via `fetch()`, which does not advertise AVIF support; real browsers negotiate AVIF/WebP and receive smaller files still, so this figure is conservative. Second, **page HTML weight did not improve - it rose slightly**. The full product catalogue is still serialised into the page because header search runs in the browser. That is a known remaining item (section 7, action 7) and was deliberately not changed before launch because it alters search behaviour. It is largely mitigated in practice: the HTML is Brotli-compressed to roughly 60 KB on the wire and is now served from the CDN rather than regenerated per request.

2. Root cause diagnosis

Cause 1 - Image optimisation was switched off (largest impact)

`components/ProductImage.jsx` passed `unoptimized` to every image whose URL was hosted on Supabase - which is every product image. Next.js therefore served the original uploaded file untouched: no resizing, no WebP/AVIF conversion, no responsive variants. Sampling the live homepage measured an average of **1,995 KB per image across 73 such images**. The correct `sizes` hints were already present on every component, so removing the flag was sufficient to activate correct responsive delivery.

Cause 2 - The CDN was bypassed on every request

The homepage, collection pages and product pages all declared `dynamic = 'force-dynamic'` with `revalidate = 0`. Live response headers confirmed the consequence: `cache-control: private, no-cache, no-store` and `x-vercel-cache: MISS` on every request, with the server re-running four Supabase queries each time. `getSiteSettings()` alone executed three times per homepage render.

Removing the flags alone was *not* sufficient for collection and product pages: without `generateStaticParams` they had no prerendered entry and continued to return `no-store`. This was caught during verification and fixed.

Cause 3 - Oversized static assets

A 240 KB favicon, a 240 KB Apple touch icon, a 240 KB app icon and a 126 KB WhatsApp icon were served byte-for-byte as authored on every page view, plus two ~2 MB hero PNGs. Favicons and touch icons never pass through the image optimiser, so their size is paid in full by every visitor.

3. Changes made - performance

Area	Change	Effect
Rendering	Replaced force-dynamic with prerendering + ISR on /, /collections/[slug], /products/[slug]; added generateStaticParams	Pages served from CDN instead of rendered per request
Cache invalidation	New lib/cache.js with tag-based purging, wired into every admin mutation (products, categories, campaigns, settings, images)	Admin edits publish immediately - no staleness trade-off
Images	Removed the unoptimized flag; enabled AVIF/WebP and a 1-year optimised-variant TTL	Full responsive optimisation on all product art
Queries	Explicit column selection instead of SELECT *; cached reads deduplicate repeated calls	Smaller result sets, far fewer round trips
Assets	New scripts/compress-static-assets.mjs; originals backed up outside public/	5,214 KB to 1,480 KB
Bundle	optimizePackageImports for lucide-react and framer-motion	Avoids pulling whole icon barrel

4. Changes made - payments (Razorpay)

The existing integration was in better shape than expected: HMAC SHA256 signature verification was already correct and timing-safe, the webhook already verified against the raw request body, and **order pricing was already validated server-side** against the database rather than trusting the client. Four defects were found and fixed.

#	Defect	Impact	Status
1	Webhook unconditionally overwrote the order's workflow status	Razorpay retries on any non-2xx. A retried delivery would revert a **shipped** order back to 'confirmed' - silent order-state corruption	**Fixed**
2	No idempotency on payment updates	The browser verify call races the webhook and both can deliver the same capture more than once	**Fixed**
3	Captured amount was never checked	A payment could be confirmed without confirming it matched the order total	**Fixed**
4	Verify endpoint wrote a failure on invalid signature	Any signed-in customer could mark **another customer's** order failed by guessing its Razorpay order id	**Fixed**

Inventory. The requested 'update inventory on payment' could not be implemented as-is: the database schema has no stock quantity column, only a `stock_status` label. A migration adds `products.stock_quantity` and an atomic `apply_order_inventory()` Postgres function, which the webhook now calls. Until that migration is run the call logs a warning and payments continue to succeed unaffected.

5. Changes made - security

Finding	Detail	Status
Admin API authorisation	All 27 API routes audited. Every admin route correctly enforces bearer-token auth with role checks. No gaps found	**Verified OK**
Unbounded admin query	Admin orders fetched every order with all line items, with no limit - cost grows without bound as orders accumulate	**Fixed**

Finding	Detail	Status
Search filter injection	Admin order search interpolated user input into a PostgREST or() filter string; commas and parentheses were parsed as filter syntax	**Fixed**
Missing security headers	Added X-Content-Type-Options, Referrer-Policy, X-Frame-Options, Permissions-Policy; noindex on admin and account routes	**Fixed**
Secret exposure	Scanned the repository for committed keys, tokens and service-role credentials	**None found**

6. Supabase advisor findings

Advisory	Level	Assessment	Action
function_search_path_mutable	WARN	Genuine. set_updated_at had a mutable search_path	**Fixed in SQL**
public_bucket_allows_listing	WARN	Genuine. The bucket is already public, so the policy added nothing for image URLs but did grant the list() API - letting anyone enumerate every file, including images for unpublished products	**Fixed in SQL**
auth_leaked_password_protection	WARN	Genuine. Dashboard-only setting, cannot be changed from SQL or code	**You must enable**
rls_enabled_no_policy x6	INFO	**Not a defect - this is the intended design.** RLS enabled with zero policies means deny-all for the public anon key. The server reaches these tables with the service-role key, which bypasses RLS. Adding policies here would **expose customer names, phone numbers, addresses and order history** to anyone holding the anon key, which ships in the browser bundle	**No action - do not 'fix'**

The six INFO advisories are the linter confirming those tables are locked down correctly. They are the expected output for this architecture, not outstanding work.

Dependency audit

Package	Severity	Assessment
next 15.5.19	High	Advisories cover Server Actions DoS and SSRF. **This codebase uses no Server Actions** (verified), so those vectors do not apply. Clearing the advisory outright requires upgrading to Next 16 - a major version change. Recommended **after** launch, not before
postcss	High	Build-time dependency only; not part of the runtime surface
sharp	High	Not a project dependency. Installed locally only to run the asset compression script; never deployed

7. Outstanding actions

LAUNCH BLOCKER - Razorpay is running TEST keys in production.

The live endpoint `/api/payments/razorpay/status` currently returns `keyMode: "test"`. The integration is complete and correct, and the checkout code places **no restriction on payment method -** so UPI, cards, netbanking and wallets will all be offered, exactly as enabled on the Razorpay account. But **real customers cannot pay until all three Razorpay keys are switched to Live mode together** after account activation. Which methods appear is governed by the Razorpay dashboard, not by this codebase.

#	Action	Why	Owner
1	Switch NEXT_PUBLIC_RAZORPAY_KEY_ID, RAZORPAY_KEY_ID and RAZORPAY_KEY_SECRET to Live together, then redeploy	Real payments are impossible on test keys. Mixed test/live keys break checkout silently	Team
2	Confirm RAZORPAY_WEBHOOK_SECRET is set and the webhook URL points to /api/webhooks/razorpay	Without it the webhook rejects every delivery, so payments captured after the browser closes never reach the order. The status endpoint now reports hasWebhookSecret so this can be checked	Team

#	Action	Why	Owner
3	Run supabase/phase2-performance-and-payments.sql in the SQL editor	Adds inventory support, the advisor fixes, a uniqueness guarantee on razorpay_order_id, and trigram indexes for admin search. Includes read-only diagnostics to run first	Team
4	Run npm run images:publish with Supabase credentials	Uploads the 89 pre-optimised WebP images (197 MB to 8.5 MB) and repoints the database. Dry-runs by default; never deletes originals, so rollback is a database update	Team
5	Enable leaked-password protection in Authentication > Providers > Email	Dashboard-only setting; cannot be scripted	Team
6	Add a paging control to the admin orders screen	The query is now bounded at 200 records. Below that nothing changes; beyond it, older orders would not be visible without paging UI	Dev
7	Move header search server-side	The full catalogue is still serialised into the page because search runs in the browser. This is the remaining page-weight item; fixing it changes search behaviour so it was not done unilaterally before launch	Dev

8. Verification performed

Production build compiles cleanly (43 static pages generated) and ESLint passes with no warnings. Changes were committed as bd90550, pushed to main and confirmed deployed to production. The following were then verified **against the live site**:

Check	Result
Homepage served from CDN	x-vercel-cache: HIT on 4 of 4 consecutive requests
All 4 security headers present	nosniff, strict-origin-when-cross-origin, SAMEORIGIN, Permissions-Policy
Product pages render	5 of 5 sampled from the live sitemap returned 200, all PRERENDER
Collection pages render	3 of 3 sampled returned 200
Sitemap coverage	160 products + 13 collections prerendered
Image optimisation	102 of 102 unique sources through /_next/image; zero raw Supabase originals remaining
Visual regression	Homepage compared before/after asset compression - no perceptible quality loss
Console / server errors	None logged

One behaviour change worth knowing. Product lookup no longer falls back to the bundled demo catalogue when the database returns no row - it now correctly returns 404. Previously a slug missing from the live database could still render a phantom product page. All 160 real products were confirmed unaffected.

Not verified. The tag-based cache purge on admin save could not be exercised, because this environment has no credentials for the Supabase project (pwdbvmplcftqnrnyizkf) - the connected account only had access to an unrelated project. Please confirm once: edit a product in the admin panel and check the storefront reflects it without a redeploy. If it does not, the fallback is that changes appear within one hour.

---

31 files changed across rendering, data access, payments, security and build configuration. Committed as bd90550, pushed to main and deployed to production. Full change detail is in the commit message. Supporting scripts: `scripts/compress-static-assets.mjs`, `scripts/publish-optimized-images.mjs`, `supabase/phase2-performance-and-payments.sql`.